Transacciones en MySQL

Optimización y Automatización de Operaciones en Bases de Datos

Transacciones

Una transacción en un Sistema de Gestión de Bases de Datos es un conjunto de órdenes que se ejecutan formando una unidad de trabajo, es decir, en forma indivisible o atómica.

- Integridad de los datos: transacciones no pueden finalizar en un estado intermedio.

Transacciones

Una transacción es un grupo secuencial de sentencias SQL.
Por ejemplo operaciones de actualización.

Una transacción sólo se confirma si cada operación individual dentro del grupo de sentencias tiene éxito.

Si al menos una de las operaciones falla, la transacción NO se completa.

ACID: Propiedades de las transacciones

- **Atomicidad:** asegura que todas las operaciones dentro de la unidad de trabajo se completen con éxito; de lo contrario, la transacción se cancela en el punto de falla y las operaciones anteriores se devuelven a su estado anterior.
- **Coherencia:** garantiza que la base de datos cambie correctamente los estados en una transacción confirmada con éxito.
- **Aislamiento:** permite que las transacciones operen independientemente y transparentes entre sí.
- **Durabilidad:** asegura que el resultado o efecto de una transacción confirmada persista en caso de una falla del sistema.

Transacciones

Inicio de transacción:

- START TRANSACTION // Inicio
- BEGIN WORK

Fin transacción:

- COMMIT // Confirma
- ROLLBACK . // Vuelve atrás

Las sentencias SQL entre las instrucciones de inicio y finalización forman parte de la transacción.

Ejemplo implementación de transacciones

```
START TRANSACTION;

    UPDATE accTable SET
    ledgerAmt=ledgerAmt-@transAmt WHERE
    customerId=1;

    UPDATE accTable SET
    ledgerAmt=ledgerAmt+@transAmt WHERE
    customerId=2;

COMMIT;
```

Transacciones

Procedimientos generales involucrados en la transacción.

- Comenzar la transacción con el comando SQL BEGIN WORK o START TRANSACTION .
- Ejecutar todas las sentencias SQL.
- Comprobar si todas las sentencias se ejecutan correctamente.
- Si la ejecución es válida, confirma mediante COMMIT; de lo contrario, revertir con el comando ROLLBACK.

Transacciones

La variable AUTOCOMMIT se establece como *verdadera* por defecto.

– Cambio a FALSE

```
SET AUTOCOMMIT=false;
SET AUTOCOMMIT=0;
```

--->Cambio a TRUE

```
SET AUTOCOMMIT=true;
SET AUTOCOMMIT=1;
```

Estado de AUTOCOMMIT

```
SELECT @@autocommit;
```

Transacciones con Control de Error

```
DECLARE handler_action HANDLER
    FOR condition_value [, condition_value] ...
    statement

handler_action:
    CONTINUE
  | EXIT
  | UNDO

condition_value:
    mysql_error_code
  | SQLSTATE [VALUE] sqlstate_value
  | condition_name
  | SQLWARNING
  | NOT FOUND
  | SQLEXCEPTION
```

Transacciones con Control de Error

Las acciones posibles que podemos seleccionar como *handler_action* son:

- `CONTINUE`: La ejecución del programa continúa.
- `EXIT`: Termina la ejecución del programa.

Transacciones con Control de Error

- **Ejemplo indicando el número de error de MySQL:**

  En este ejemplo estamos declarando un *handler* que se ejecutará cuando se produzca el error `1051` de MySQL, que ocurre cuando se intenta acceder a una tabla que no existe en la base de datos. En este caso la acción del *handler* es `CONTINUE` lo que quiere decir que después de ejecutar las instrucciones especificadas en el cuerpo del *handler* el procedimiento almacenado continuará su ejecución.

  ```
  DECLARE CONTINUE HANDLER FOR 1051

  BEGIN

    -- body of handler

  END;
  ```

Transacciones con Control de Error

Ejemplo para `SQLSTATE`:

También podemos indicar el valor de la variable `SQLSTATE`. Por ejemplo, cuando se intenta acceder a una tabla que no existe en la base de datos, el valor de la variable `SQLSTATE` es `42S02`.

```
DECLARE CONTINUE HANDLER FOR SQLSTATE '42S02'
BEGIN
  -- body of handler
END;
```

Transacciones con Control de Error

Ejemplo para `SQLWARNING`:

Es equivalente a indicar todos los valores de `SQLSTATE` que empiezan con `01`.

```
DECLARE CONTINUE HANDLER FOR SQLWARNING
BEGIN
  -- body of handler
END;
```

Transacciones con Control de Error

Ejemplo para `NOT FOUND`:

Es equivalente a indicar todos los valores de `SQLSTATE` que empiezan con `02`. Lo usaremos cuando estemos trabajando con cursores para controlar qué ocurre cuando un cursor alcanza el final del *data set*. Si no hay más filas disponibles en el cursor, entonces ocurre una condición de `NO DATA` con un valor de `SQLSTATE` igual a `02000`. Para detectar esta condición podemos usar un *handler* para controlarlo.

```
DECLARE CONTINUE HANDLER FOR NOT FOUND
BEGIN
  -- body of handler
END;
```

Transacciones con Control de Error

Ejemplo para `SQLEXCEPTION::`

Es equivalente a indicar todos los valores de `SQLSTATE` que empiezan por `00`, `01` y `02`.

```
DECLARE CONTINUE HANDLER FOR SQLEXCEPTION
BEGIN
  -- body of handler
END;
```

Transacciones con Control de Error

```
DELIMITER //
create procedure pro1(_clave integer, _dato integer)
BEGIN
 -- Declare acción por error
 DECLARE EXIT HANDLER FOR SQLEXCEPTION
 BEGIN
        rollback;
        SELECT 'Error en transaction' AS message;
 END;
        START TRANSACTION;
        update a set z = _dato where c = 1;
        insert into a(c,z) values (_clave,_dato);
        commit;
 end //
DELIMITER ;
```

Cursores

```
DROP PROCEDURE IF EXISTS cursores;

DELIMITER //

CREATE PROCEDURE cursores ()
BEGIN
-- Declaramos variables que almacenan los datos del select especificado
 DECLARE v_id integer;
 DECLARE v_titulo varchar(100);
 DECLARE v_estudio varchar(64);
 DECLARE v_anio integer;
 DECLARE v_precio double(5,2);
 DECLARE v_tipo varchar(20);

-- Variable para controlar el fin del ciclo de recorrido del conjunto select
 DECLARE fin INTEGER DEFAULT 0;

-- Declaramos cursor a utilizar
 DECLARE cursor1 CURSOR FOR
  SELECT id, titulo, estudio, anio, precio, tipo
  FROM programas;
```

Cursores

```
-- Declaramos Condición de salida
 DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin=1;

 OPEN cursor1;
 ciclo: LOOP
        FETCH cursor1 INTO v_id, v_titulo, v_estudio, v_anio, v_precio, v_tipo;
        IF fin = 1 THEN
          LEAVE ciclo;
        END IF;

        SELECT v_id, v_titulo, v_estudio, v_anio, v_precio, v_tipo;

 END LOOP ciclo;

 CLOSE cursor1;
END //
DELIMITER ;
```